

MP468C — Computational Physics 2 — Lab 01 — some
comments / answers

1. Plot the normalized distribution function of the quantity

$$y = -x^3 \ln x$$

where x is a uniformly distributed random number between 0 and 1. Use at least 100 bins in your plot (histogram), and find how many random numbers you need to get a smooth curve.

Plotting questions:

- (a) How do you control the number of bins when using matplotlib to plot histograms?
- (b) Matplotlib has an option of normalizing histograms. This option multiplies the histogram counts by some factor. What factor? Should the bar heights add to 1 when you normalize, or should the areas of the bars add to 1? Explain.
- (c) Is there any advantage to using the seaborn package for plotting histograms?

More important question:

Can you derive the functional form of the probability distribution (pdf) that you obtain? If so, don't we want this function plotted on top of the normalized histogram?

==--==--==--==--

Discussion/answers →

Discussion concerning part (b):

What is an un-normalized histogram?

If the total number of data points is N_{count} , then the sum of all heights in the un-normalized histogram is N_{count} , because each height represents the number of data points in that bin. If the bins each have equal width h , then the area under the un-normalized histogram is hN_{count} . (If the bins have unequal width, you need a more complicated expression.)

What does a normalized histogram mean?

Here are two reasonable interpretations:

[a] that the height of each bin represents the probability of finding numbers in that interval;

[b] that the histogram approximates a probability distribution function.

In case [a], the sum of heights of the bins should be equal to 1. So the un-normalized histogram heights have to be divided by a factor of N_{count} .

In case [b], the area under the histogram should be equal to 1. So the un-normalized histogram heights have to be divided by a factor of hN_{count} .

With `density=True`, matplotlib implements case [b].

Discussion concerning the “more important question”:

We are given

$$y = f(x) = -x^3 \ln x$$

which means that $x = f^{-1}(y)$ is difficult to calculate analytically.

We are given

$$P_X(x) = \begin{cases} 1 & \text{for } x \in [0, 1] \\ 0 & \text{for } x \notin [0, 1]. \end{cases}$$

and we have to find the probability distribution $P_Y(y)$ of the variable $Y = -X^3 \ln X$.

The probability of finding X in an interval $(x, x + dx)$ should be the same as finding Y in the corresponding interval $(y, y + dy)$. This can be expressed as

$$|P_X(x)dx| = |P_Y(y)dy| \quad \implies \quad P_Y(y) = \frac{P_X(x)}{|dy/dx|} = \frac{P_X(x)}{|f'(x)|}$$

The absolute value is there because y might be a decreasing function of x and we don't want to end up with a negative probability distribution. Taking the derivative of the function $f(x)$ gives us

$$P_Y(y) = \frac{P_X(x)}{|x^2(3 \ln x + 1)|} = \begin{cases} \frac{1}{|x^2(3 \ln x + 1)|} & \text{for } x \in [0, 1] \\ 0 & \text{for } x \notin [0, 1]. \end{cases}$$

This is a fine expression, but it is expressed in terms of x . All we have to do now is to express x in terms of y .

Unfortunately, we don't know the functional form of $x = f^{-1}(y)$, so our analytical derivation stops in failure at this point. Sad!

If we had the functional form, we could also solve $f^{-1}(y) \in [0, 1]$ to find out the interval of y for which $P_Y(y)$ is nonzero. From your histograms, this range of values of y is seen to be between 0 and 0.123 approximately. If you plot the function $f(x)$, you should see why this is so. Pity we can't do this analytically! But this is a good lesson — most problems cannot be solved analytically.

2. Generate pairs of random numbers (x, y) , each uniformly distributed random number between 0 and 1. Compute $R = \sqrt{x^2 + y^2}$. Using at least 10^7 pairs, find the proportion p of pairs satisfying $R < 1$, and print $4p$. What do you get?

Explain geometrically the value of $4p$.

==--==--==--==--

Discussion/answers →

If you interpret the pair of random numbers as describing a 2D Euclidean coordinate, then these points fall on the square shaped region $x \in [0, 1], y \in [0, 1]$, of area 1.

If we think of a circle centered at the origin, the part of the circle in the first quadrant encloses a region of the square which has area $\frac{1}{4}\pi(1)^2 = \pi/4$.

Hence, you expect that the proportion of randomly chosen points in the square which fall inside the quarter-circle is $\pi/4$. Thus, if you calculate the proportion in the question using a large number of uniformly distributed (x, y) pairs, and then multiply it by 4, you should get π .

Here's a slightly modified version of the code one of you submitted:

```
numpairs=10**7
count=0
x=np.random.rand(2,numpairs)
# pairs of random numbers, stored in a 2xnumpairs array
R=np.zeros(numpairs)
for i in range(numpairs):
    R[i]=np.sqrt(x[0,i]**2+x[1,i]**2)
    if(R[i]<1):
        count+=1
prop=float(count)/numpairs # Proportion of pairs with R<1
print("Approximation for pi: "+str(4*prop))
```

Note: I used `float(count)/numpairs` instead of just `count/numpairs`, so that the code would run with python2 as well. In python2, `count/numpairs` would give zero, integer division! In python3, the conversion to float is not necessary.

There are some inefficiencies in the code above. First, why create a whole array of R ? Only one value of R is used in every step. The following should be faster and more memory-efficient when `numpairs` is large.

```
numpairs=10**7
count=0
x=np.random.rand(2,numpairs)
# pairs of random numbers, stored in a 2xnumpairs array
for i in range(numpairs):
    R=np.sqrt(x[0,i]**2+x[1,i]**2)
    if(R<1):
        count+=1
prop=float(count)/numpairs # Proportion of pairs with R<1
print("Approximation for pi: "+str(4*prop))
```

I got rid of the array, and only have a variable R .

Thinking further about efficiency, you don't actually need to calculate the square root: $\sqrt{x^2 + y^2} < 1$ is the same condition as $x^2 + y^2 < 1$, and in fact you don't even need to define a variable R . (Since I formulated the question in terms of $R = \sqrt{x^2 + y^2}$, I take blame for this inefficiency.) Writing

```
numpairs=10**7
count=0
x=np.random.rand(2,numpairs)
# pairs of random numbers, stored in a 2xnumpairs array
for i in range(numpairs):
    if(x[0,i]**2+x[1,i]**2 <1):
        count+=1
prop=float(count)/numpairs # Proportion of pairs with R<1
print("Approximation for pi: "+str(4*prop))
```

saves time by avoiding calculating square roots 10 million times (probably a major gain), and maybe also by avoiding assigning values to a variable 10 million times (probably a minor gain).

It seems to me that there is a more serious issue to be addressed. We created `numpairs` ($= 10^7$) pairs of floating-point numbers, and then used them only one by one. The whole time, while the program is running, our computer memory (RAM) is hogged by 20 million floats which we are only using pair by pair. If each takes 64 bits (8 bytes), the number of bytes is $2 \times 10^7 \times 8$, i.e., 0.16 GB. A laptop bought in the last 10 years will generally have at least 2GB of memory, so maybe that's not too bad; your laptop won't crash. But what if you wanted to find a more accurate approximation for π , and wanted to use 10^8 or 10^9 pairs? You might be straining or crashing your laptop. So, here's an attempt of being memory-efficient:

```
numpairs=10**7
count=0
for i in range(numpairs):
    x = np.random.rand(2) # a single pair of random numbers
    if(x[0]**2+x[1]**2<1):
        count+=1
prop=float(count)/numpairs # Proportion of pairs with R<1
print("Approximation for pi: "+str(4*prop))
```

You should be able to run this for 10^8 or maybe even 10^9 pairs. (It just takes a very long time, but nothing to be done about that. You probably want to run such long calculations overnight or on remote machines.)

When I try to run it on my desktop machine for 10^{10} pairs, `spyder/python` refuses, and says

```
for i in range(numpairs):
MemoryError
```

There is probably some way to rewrite the for loop, but we have improved our program enough for now, I think.